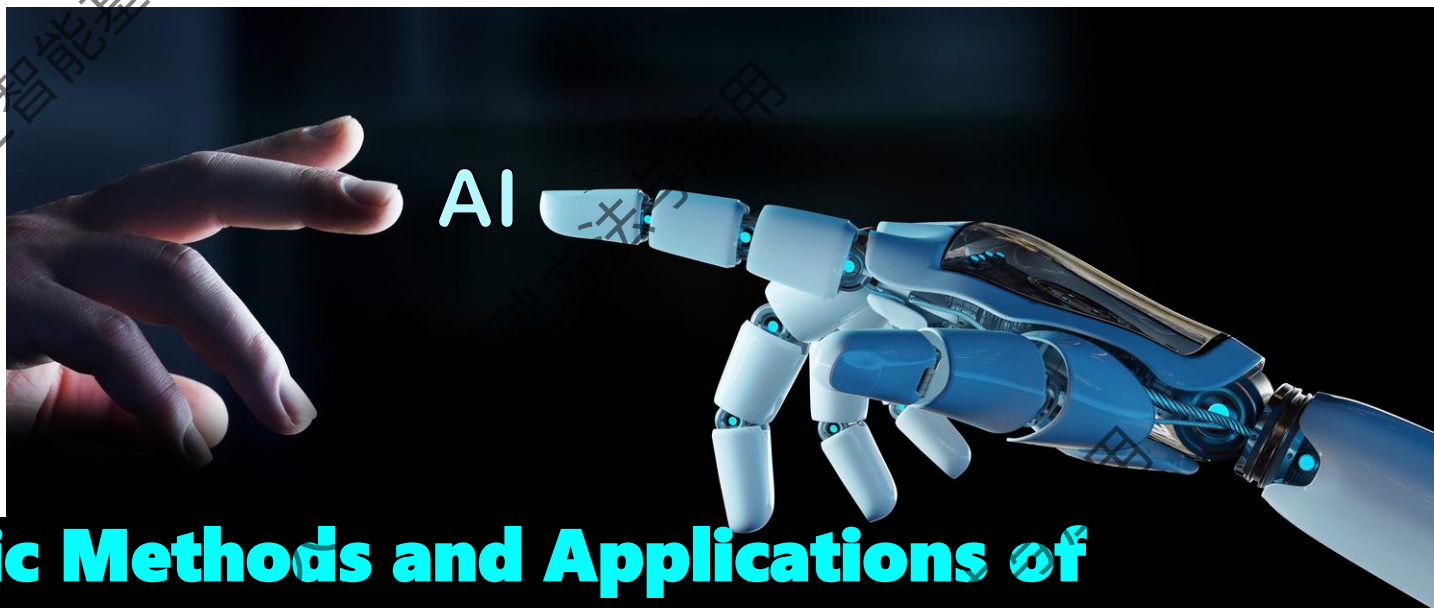




群聊：人工智能基础与应用I-06班



该二维码于2023年10月7日前有效，重新进入将更新



Basic Methods and Applications of Artificial Intelligence

——搜索求解（以自动驾驶视角）

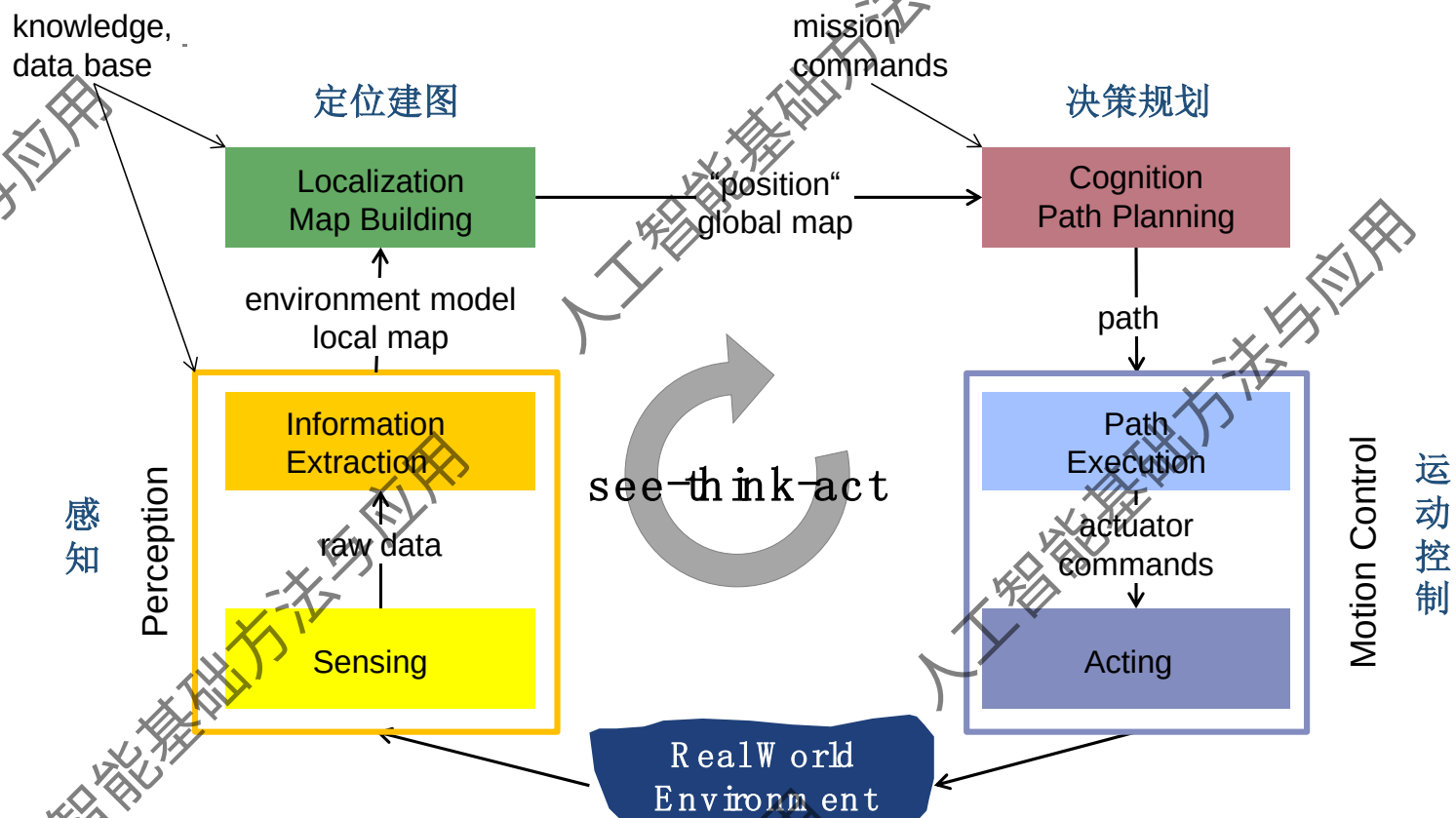
人工智能学院 / 人工智能与机器人研究所
人机混合增强智能全国重点实验室
陈炜煌



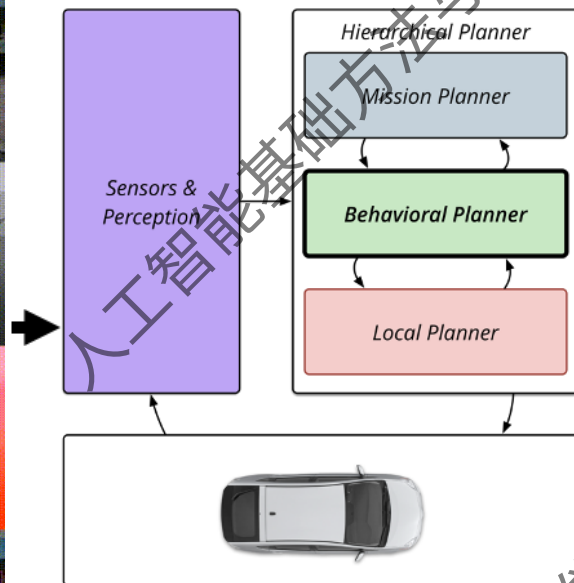
IAIR Est. 1986
Institute of
Artificial Intelligence
and Robotics, Nankai University

一、自动驾驶概况

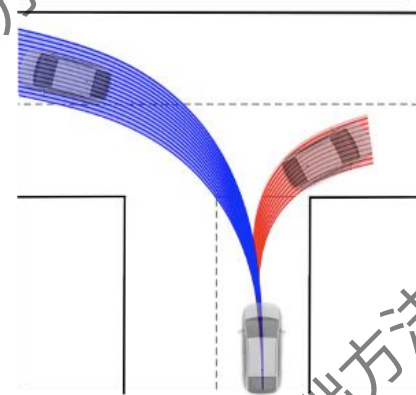
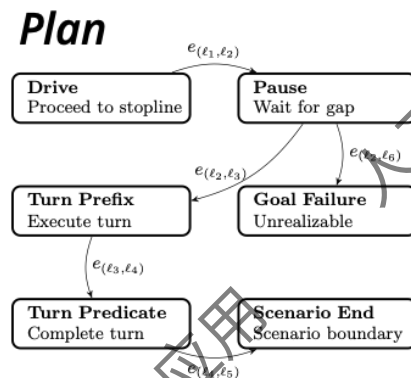
传统模块化自动驾驶技术框架



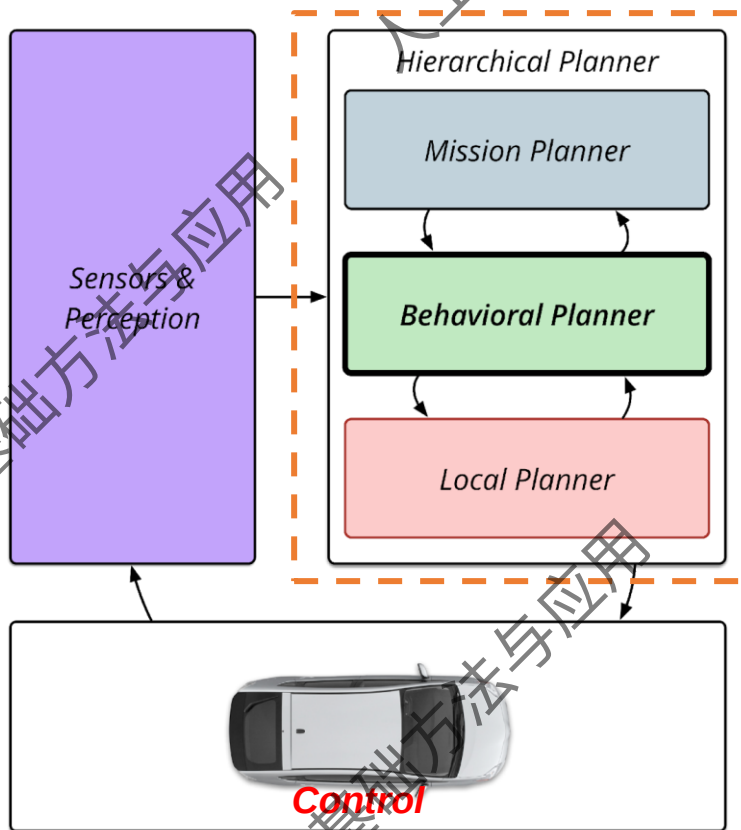
从视觉信息到驾驶动作



Everything combined:
a function that generates a
sequence of *steering* and
acceleration inputs...



三级规划框架



Mission Planner:

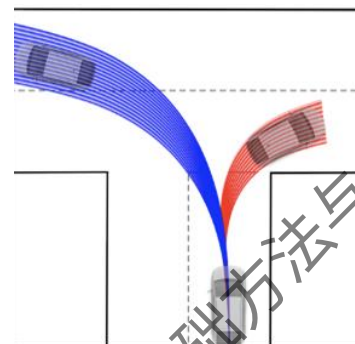
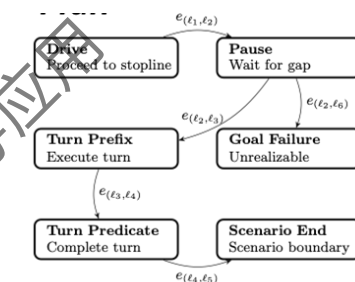
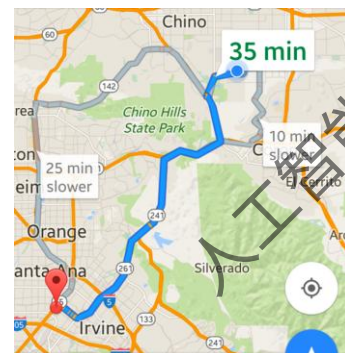
What is the overall goal of the vehicle?

Behavioral Planner:

What rules should the vehicle follow in different situations?
Makes discrete choices.

Local Planner:

What is the optimal trajectory from position to a goal?



When planning is easy



When planning is not easy



西安交大无人驾驶发展历程



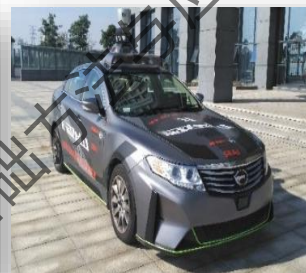
SpringRobo
t
2002~2009



夸父I号(2010~)



挑战号
(2014~)



新挑战号(2016)



新挑战号(2017)



夸父号(2009
年)



夸父II号
(2011~2013)



新夸父I号
(2016~)



发现号(2017~)



奋进号(2017~)



Pioneer 先锋号无人驾驶平台

视觉信息处理与应用国家工程实验室
人工智能与机器人研究所
西安交通大学

二、全局路径规划

全局路径规划

- 根据车辆起点、目标点和地图信息，在路网中搜索一条可行且代价较低的路线
- 主要输入：车辆当前位置、目标位置、高精度地图或道路拓扑、交通规则和实时路况
- 主要输出：全局参考路线，为行为决策和局部规划提供方向与约束
- 目标：在满足连通性、通行规则和安全约束的前提下，尽量降低行驶代价



把真实道路抽象成路网

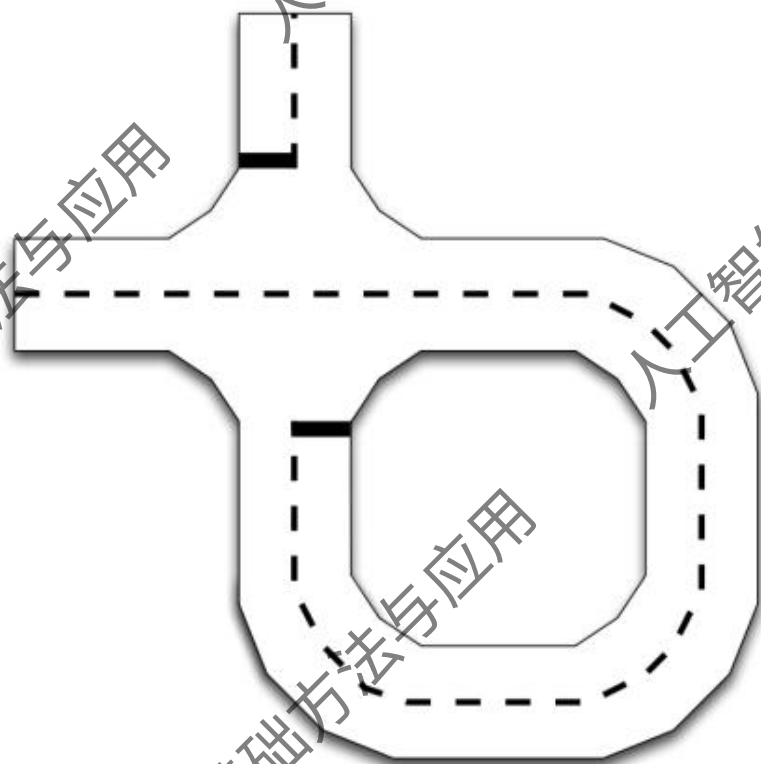


真实道路

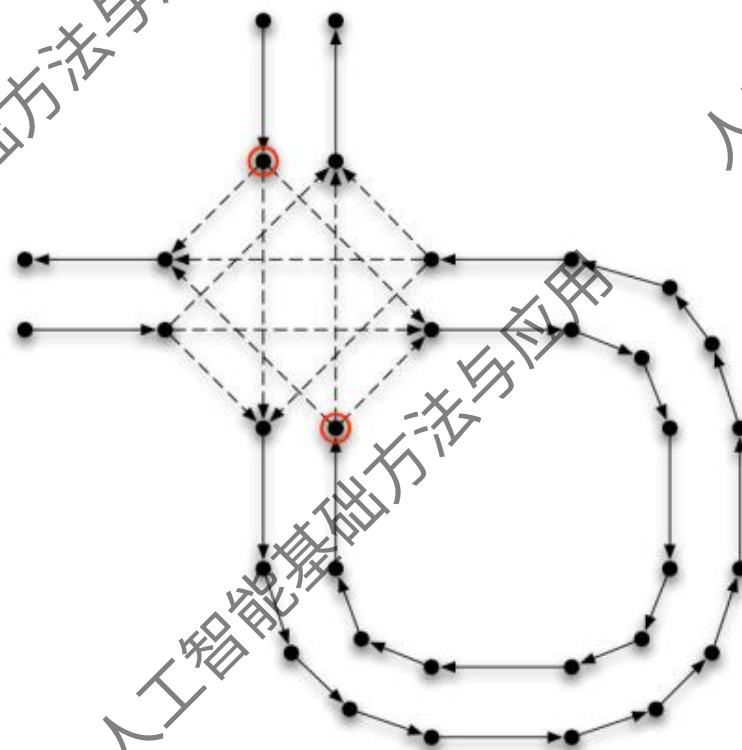


道路网络

道路网络=图结构



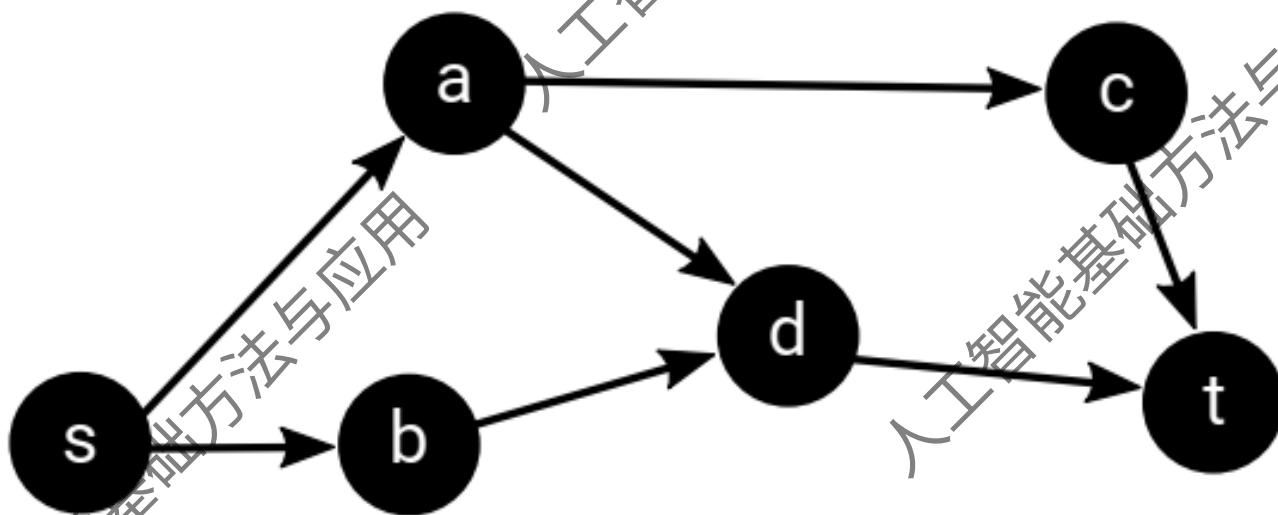
带转弯和环形部分的道路形状



节点和有向连接

道路网络=图结构

- 道路网络可以表示为带权有向图 $G = (V, E)$ ，其中：
 - $v \in V$ 表示路口或车道节点
 - $(u, v) \in E$ 边表示道路路段或车道段
 - 边权可以表示距离、时间、能耗或综合通行代价



s表示起点, t表示终点

三、搜索求解一把路径规划 写成搜索问题



为什么路径规划需要搜索？

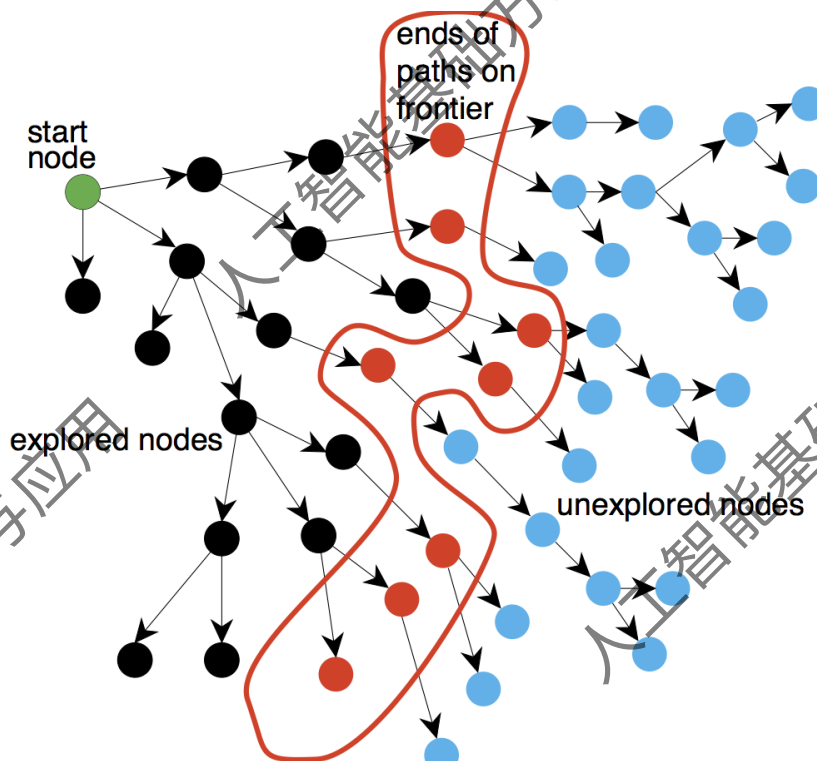
- 路径规划 = 在大量候选路线中寻找合适的路线
- 从起点到终点通常存在**大量候选路径**
- 搜索需要回答三个问题
 - What: 搜索什么?
 - Where: 在哪里搜索
 - How: 怎么搜索?



找到解 + 少走弯路 + 资源消耗少

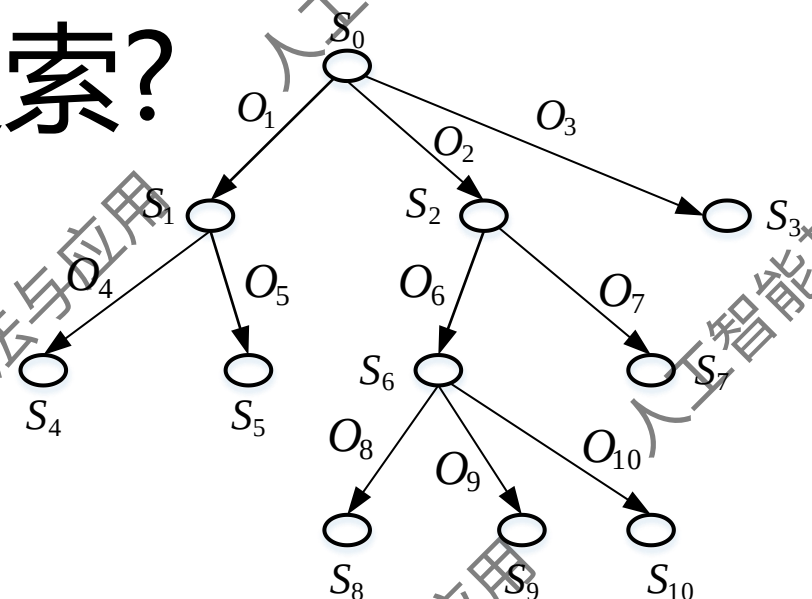
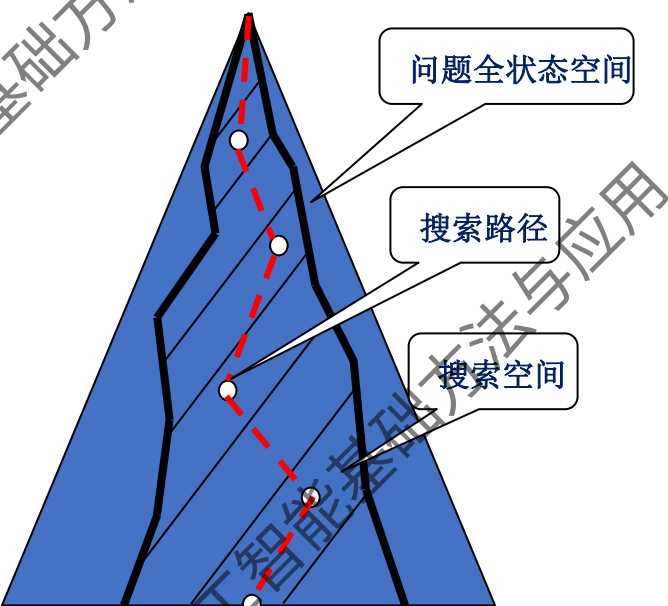
What: 搜索什么?

- 一条总代价比较小, 甚至最小的路径

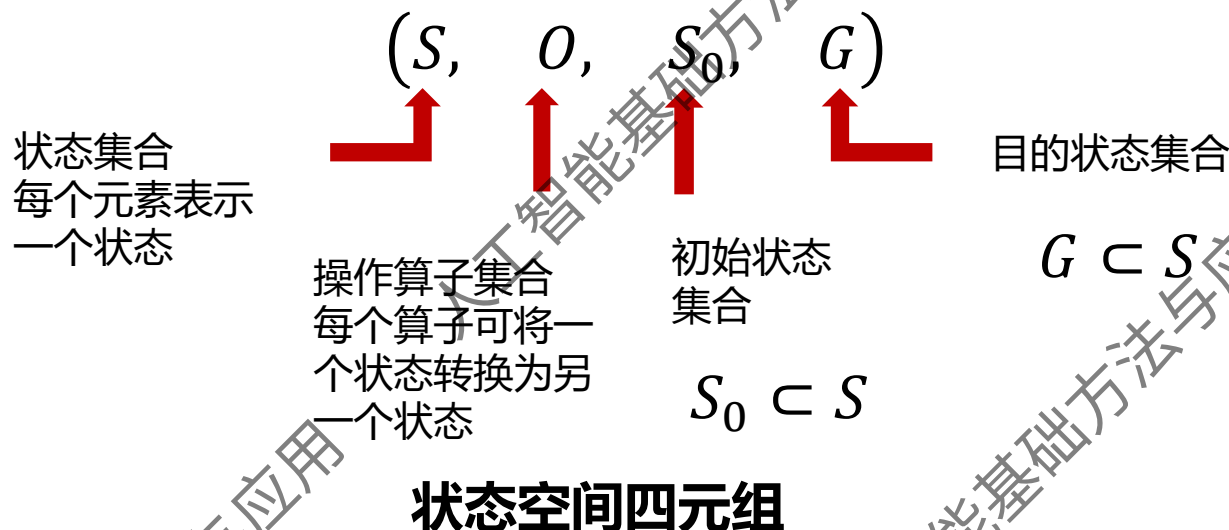


Where: 在哪里搜索?

- 搜索空间：所有可能被考虑到的状态的集合
- 状态：车辆当前所处的位置以及相关约束条件

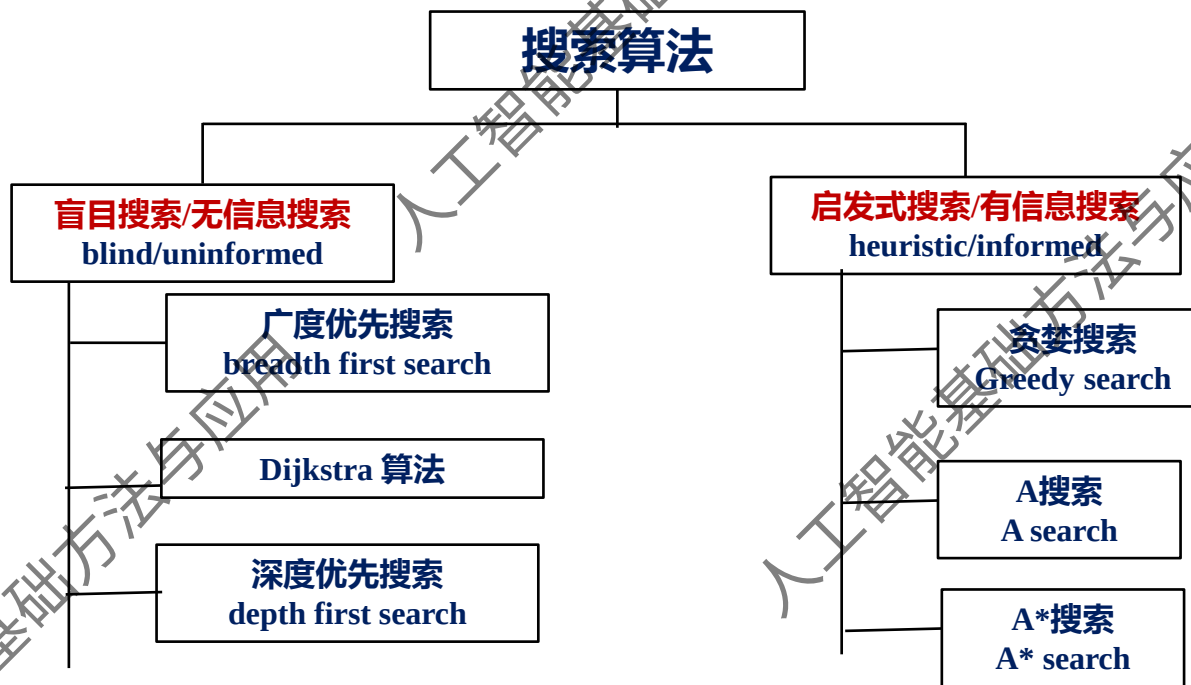


状态空间图



How: 怎么搜索?

- 盲目搜索（无信息搜索）：只利用问题本身的预设规则
- 启发式搜索（有信息搜索）：利用额外的知识进行搜索



不预测哪条路
更有希望

带目标估计的
预测

目标：少花时间，少占内存

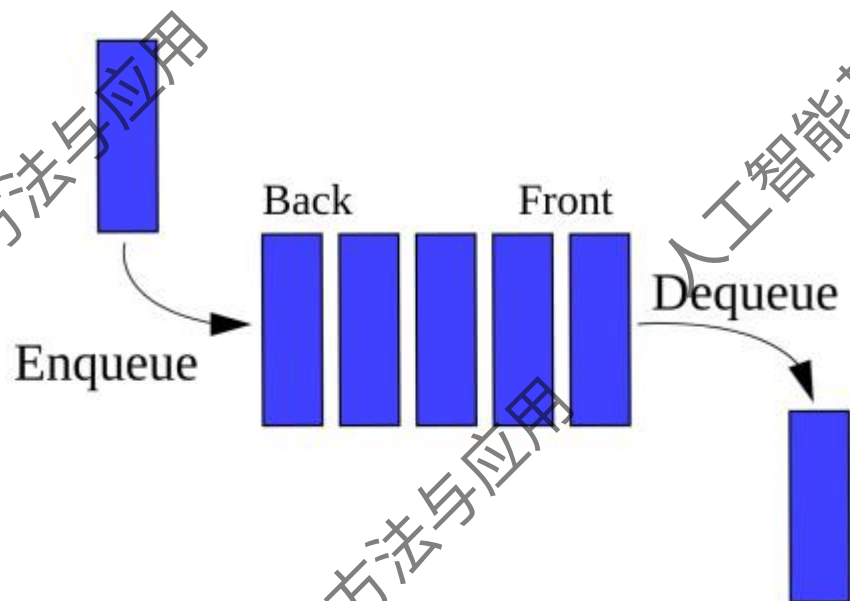
搜索求解的基本概念

- 状态（节点）：描述环境状况的表达
- 动作（边）：从当前所处的状态转移到下一时刻的状态所进行的操作
- 状态转移：对某一时刻对应状态执行动作所达到的状态
- 路径：由一系列动作所连接的状态序列
- 代价：评估一条路径所消耗资源的数值度量（如距离、步数、时间、能量等）
- 目标测试：评估当前状态是否为所解的目标状态

四、广度优先搜索

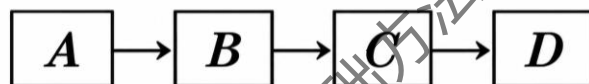
队列

- 队列：由一组按顺序排列的元素构成的数据结构
- 队列遵循**先进先出**原则，即 FIFO (First In, First Out)
- 队列通常可以使用**链表**实现
- 在合理的数据结构实现下，入队和出队操作的**时间复杂度**均为 $O(1)$



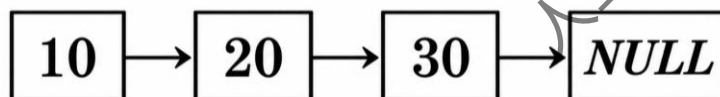
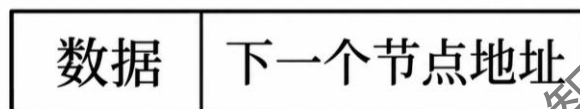
链表

- 链表：按顺序连接相邻数据的数据结构



- 每个节点通常包含两部分：

- 数据域：保存当前数据，比如 A、B、C
- 指针域：保存下一个节点的位置



广度优先搜索

- 广度优先搜索 (Breadth First Search, BFS)
- 基本思想：优先扩展同级直接相连的节点
- 以接近起始节点的程度依次扩展节点、**逐层搜索**



广度优先搜索

- 适用于**无权图**的算法，能够得到最优解
- **时间复杂度**为： $O(|V| + |E|)$

Algorithm 1 BFS(G, s, t)

```
1: open  $\leftarrow$  Queue(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.enqueue(s)
3: while !open.isEmpty() do
4:   u  $\leftarrow$  open.dequeue()
5:   if u = t then
6:     return extractPath(u, predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed and v  $\notin$  open then
9:       open.enqueue(v)
10:      predecessors[v]  $\leftarrow$  u
11:   closed.add(u)
```

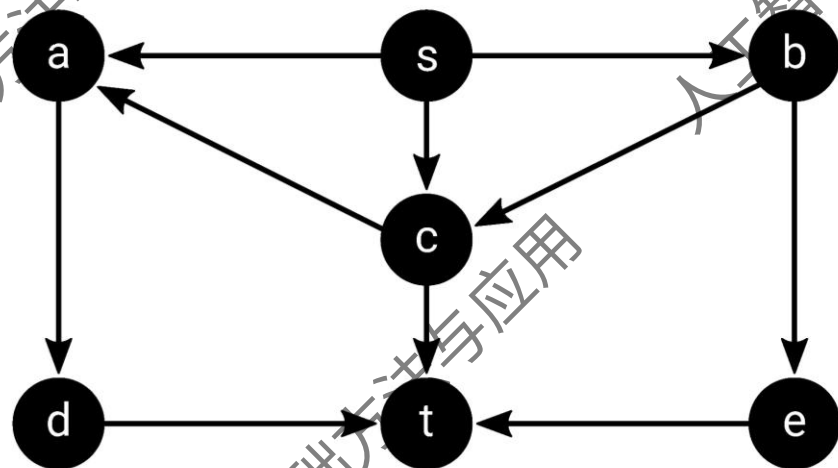


- 开集 (Open Set) : 保存已经被发现、但还没有被扩展处理的节点
- 闭集 (Closed Set) : 保存已经被取出并完成扩展的节点
- 前驱字典: 记录每个节点是从哪个节点到达的
- 入队 (Enqueue) : 在队列的末端 (队尾) 加入元素
- 出队 (Dequeue) : 从队列的前端 (队首) 移除元素

示例1

Algorithm 1 BFS(G, s, t)

```
1: open  $\leftarrow$  Queue(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.enqueue(s)
3: while open.isEmpty() do
4:    $u \leftarrow$  open.dequeue()
5:   if  $u = t$  then
6:     return extractPath( $u$ , predecessors)
7:   for all  $v \in u.\text{successors}()$  do
8:     if  $v \notin \text{closed}$  and  $v \notin \text{open}$  then
9:       open.enqueue( $v$ )
10:      predecessors[ $v$ ]  $\leftarrow u$ 
11:   closed.add( $u$ )
```



Open Queue:

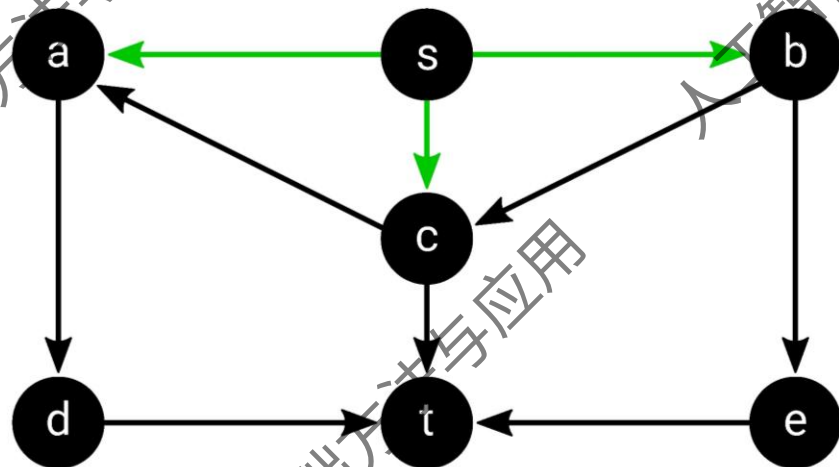
s

Closed Set:

示例1

Algorithm 1 BFS(G,s,t)

```
1: open  $\leftarrow$  Queue(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.enqueue(s)
3: while open.isEmpty() do
4:    $u \leftarrow$  open.dequeue()
5:   if  $u = t$  then
6:     return extractPath(u,predecessors)
7:   for all  $v \in u.\text{successors}()$  do
8:     if  $v \notin \text{closed}$  and  $v \notin \text{open}$  then
9:       open.enqueue(v)
10:      predecessors[v]  $\leftarrow$  u
11:   closed.add(u)
```



Open Queue:

a b c

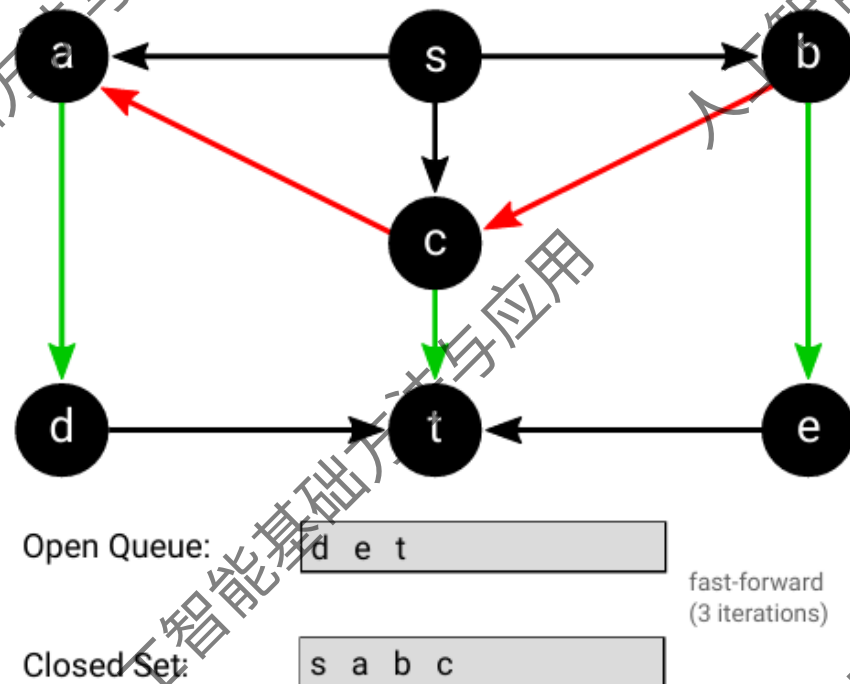
Closed Set:

s

示例1

Algorithm 1 BFS(G,s,t)

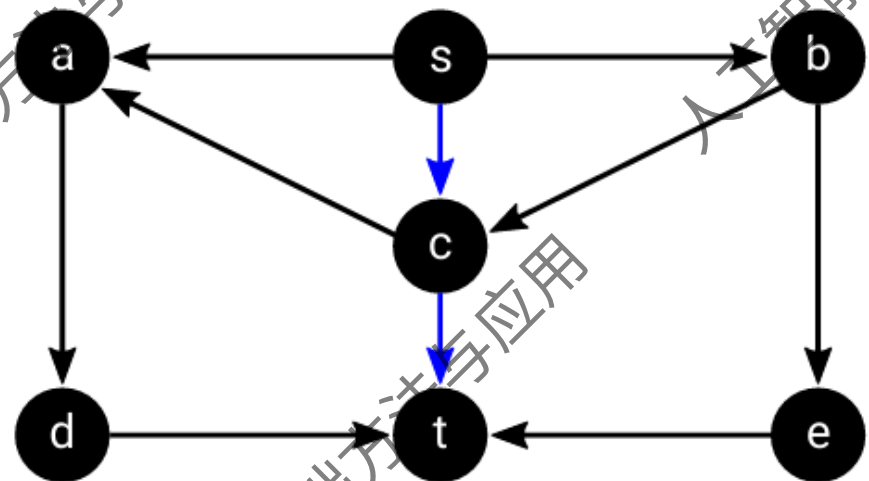
```
1: open  $\leftarrow$  Queue(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.enqueue(s)
3: while open.isEmpty() do
4:   u  $\leftarrow$  open.dequeue()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed and v  $\notin$  open then
9:       open.enqueue(v)
10:      predecessors[v]  $\leftarrow$  u
11:   closed.add(u)
```



示例1

Algorithm 1 BFS(G,s,t)

```
1: open  $\leftarrow$  Queue(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.enqueue(s)
3: while !open.isEmpty() do
4:   u  $\leftarrow$  open.dequeue()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed and v  $\notin$  open then
9:       open.enqueue(v)
10:      predecessors[v]  $\leftarrow$  u
11:  closed.add(u)
```



Final Path:

s c t

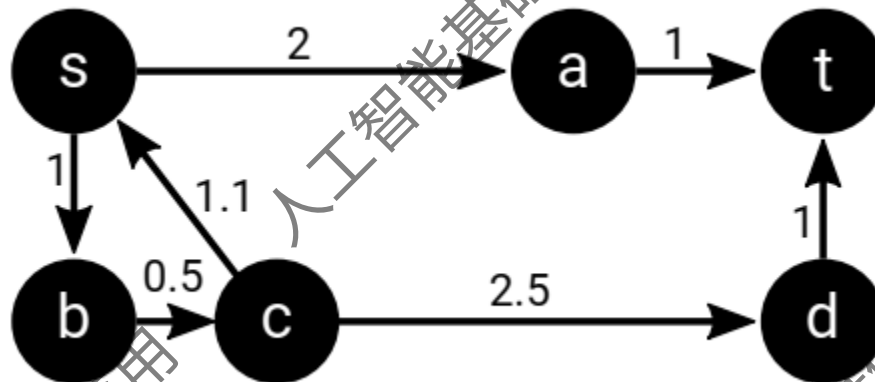
Closed Set:

s a b c d e

fast-forward
(3 iterations)

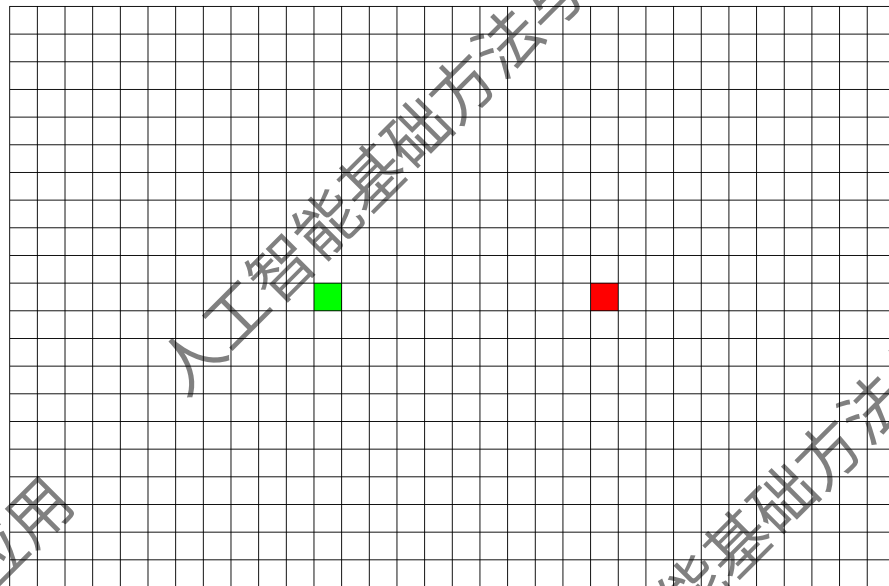
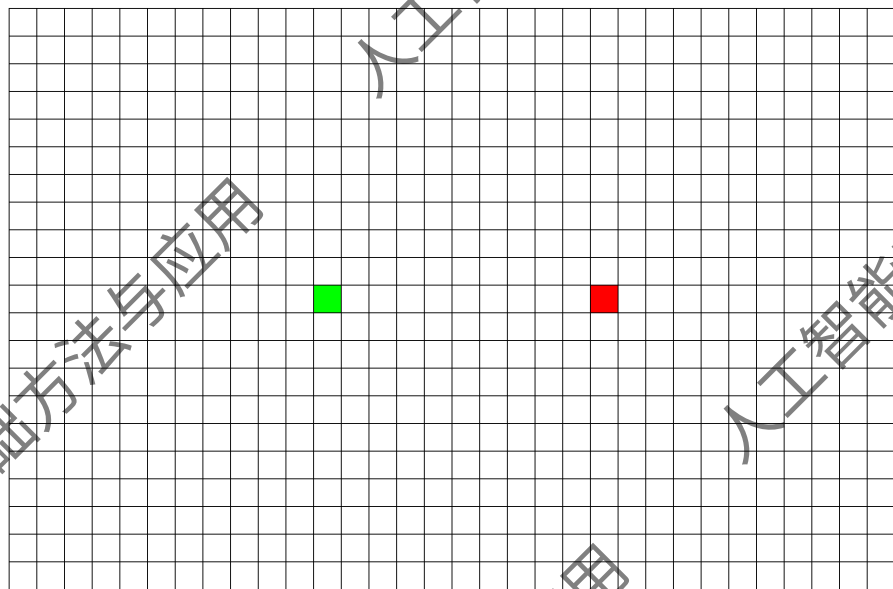
BFS忽略了路径长度

- 问题：无权图无法准确表示长度不同的道路路段
- 解决方案：使用加权图表示道路网络



- 图中的每条边都具有相应的权重，该权重可以表示道路路段的长度或车辆通过该路段所需的行驶时间

路径规划一般不用DFS算法



五、Dijkstra最短路径算法



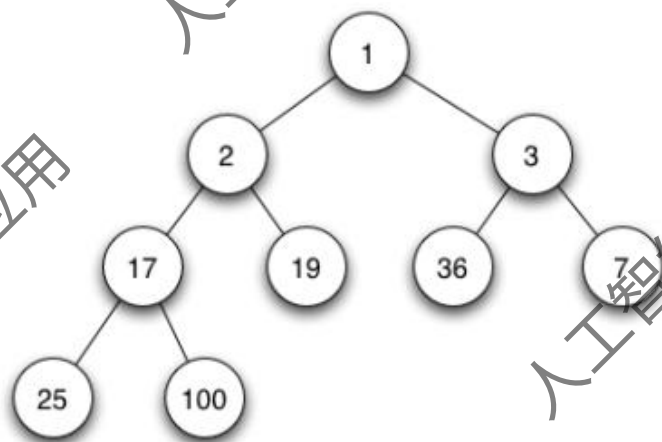
Dijkstra算法设计起因和过程

“What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city. It is the algorithm for the shortest path, which I designed in about twenty minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path. As I said, it was a twenty-minute invention. In fact, it was published in 1959, three years later. The publication is still readable, it is, in fact, quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities. Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame.”

- Edsger Dijkstra, in an interview with Philip L. Frana, Communications of the ACM, 2001

最小堆 (Min Heap)

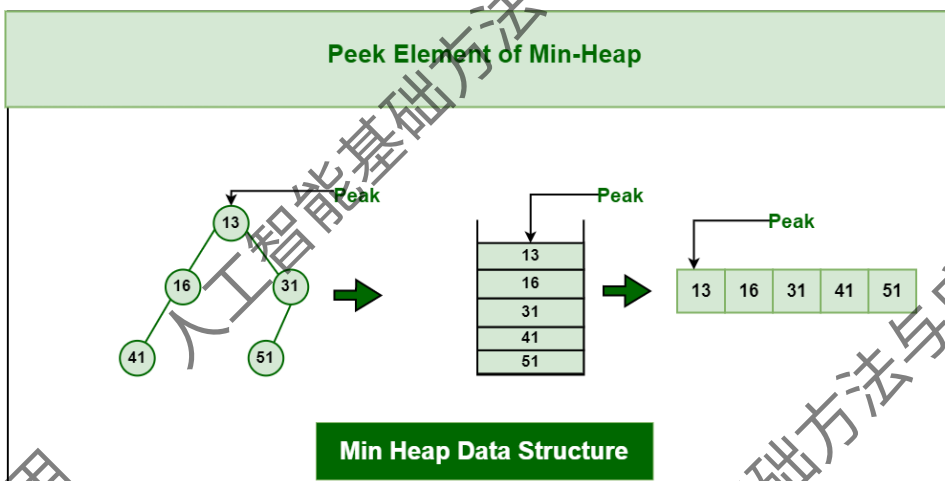
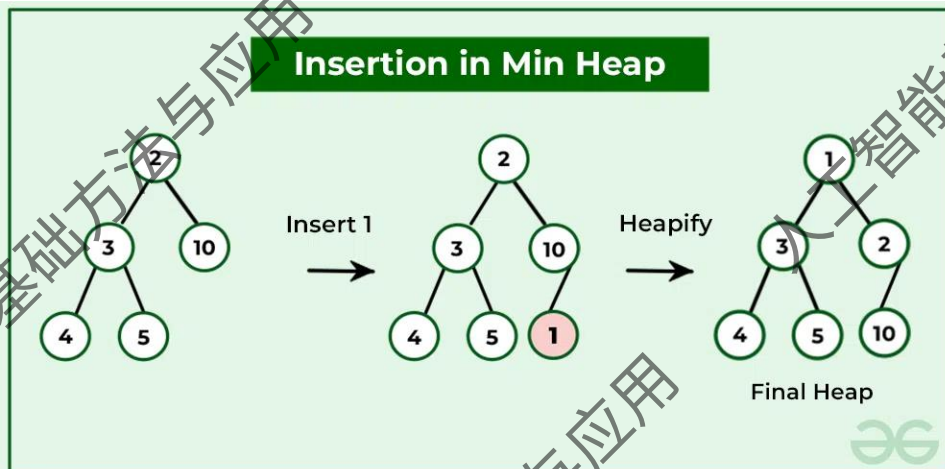
- Heap = Tree
- Min Heap是一种可高效实现**优先队列**的**树形**数据结构
- 堆中的元素具有**局部有序性**
- 实现方式: **完全二叉树**



完全二叉树

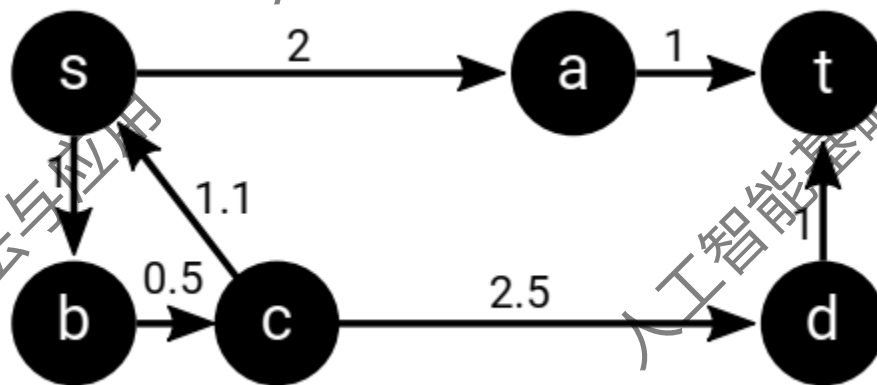
最小堆的基本操作

- 插入 Insert
- 删除最小值 Delete-Min
- 获取最小值 Get-Min
- 建堆 Build-Heap



Dijkstra最短路径算法

- 功能：在有权图中求解最短路径问题。
 - 在有权图中给出起点出发，到达目标点所经过边的权重和最小的一条路径
- 输入：有权图、指定目标节点
- 输出：起点到目标点的路径及权重



Dijkstra最短路径算法

Algorithm 1 BFS(G,s,t)

```
1: open  $\leftarrow$  Queue(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.enqueue(s)
3: while !open.isEmpty() do
4:   u  $\leftarrow$  open.dequeue()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed and v  $\notin$  open then
9:       open.enqueue(v)
10:      predecessors[v]  $\leftarrow$  u
11:   closed.add(u)
```

Algorithm 2 Dijkstra(G,s,t)

```
1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost+uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
```

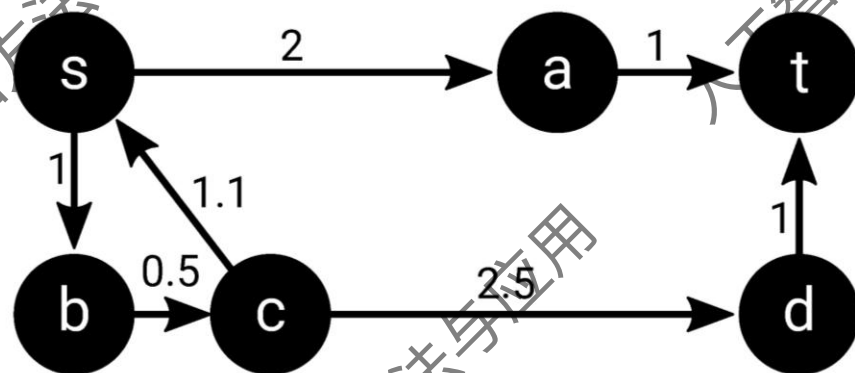
- 该算法能够得到最优解，其时间复杂度为 $O(|V| \log |V| + |E|)$ ，其中， $|V|$ 表示顶点数量， $|E|$ 表示边的数量

示例2

Algorithm 2 Dijkstra(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
  
```



Open Min Heap:

Node	s
Cost to Vertex	0

Closed Set:

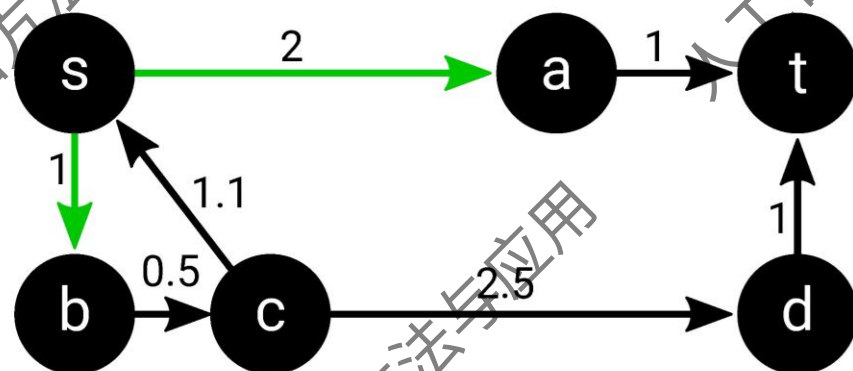
--

示例2

Algorithm 2 Dijkstra(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
  
```



Open Min Heap:

Node	b	a
Cost to Vertex	1	2

Closed Set:

s

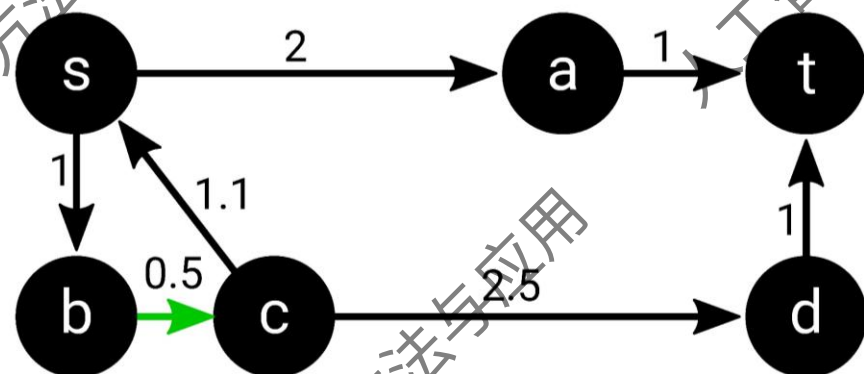


示例2

Algorithm 2 Dijkstra(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
  
```



Open Min Heap:

Node	c	a
Cost to Vertex	1.5	2

Closed Set:

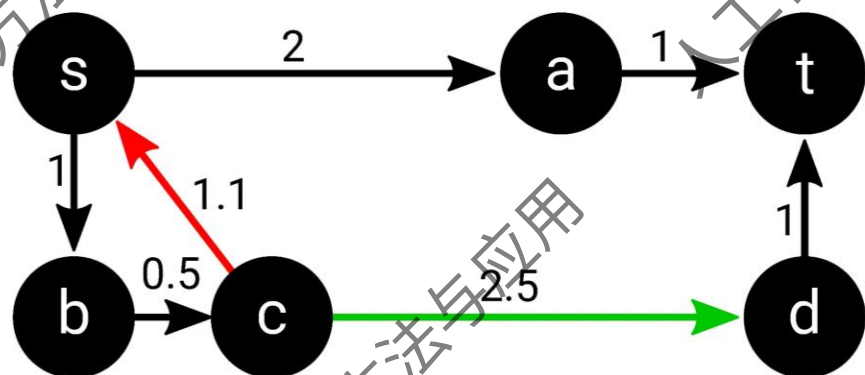
s	b
---	---

示例2

Algorithm 2 Dijkstra(G, s, t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s, 0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u, predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u, v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v, uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
  
```



Open Min Heap:

Node	a	d
Cost to Vertex	2	4

Closed Set:

s	b	c
---	---	---

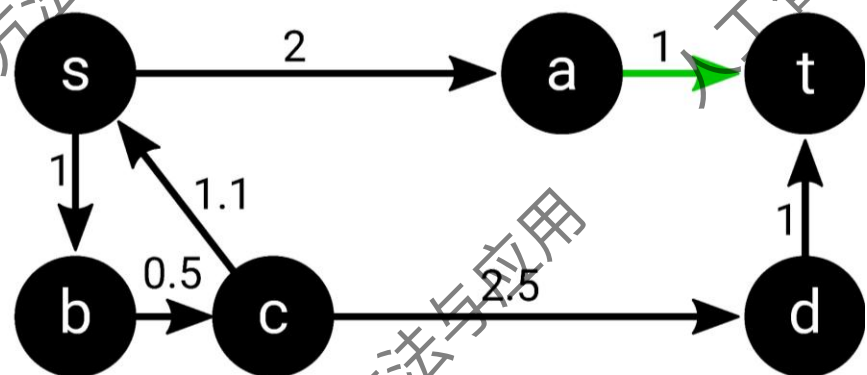


示例2

Algorithm 2 Dijkstra(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
  
```



Open Min Heap:

Node	t	d
Cost to Vertex	3	4

Closed Set:

s	b	c	a
---	---	---	---

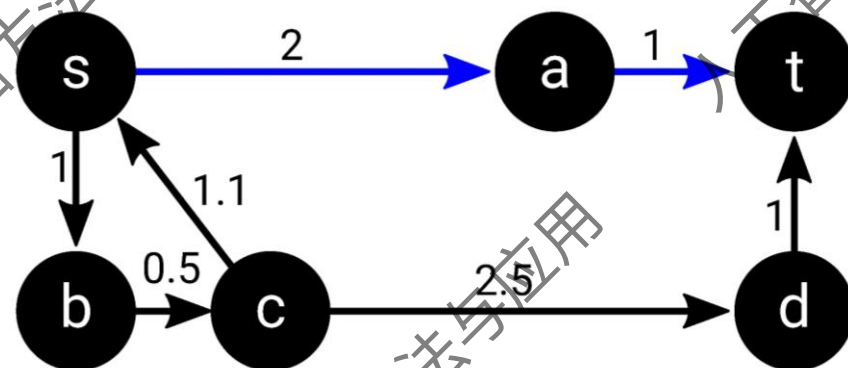


示例2

Algorithm 2 Dijkstra(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
  
```



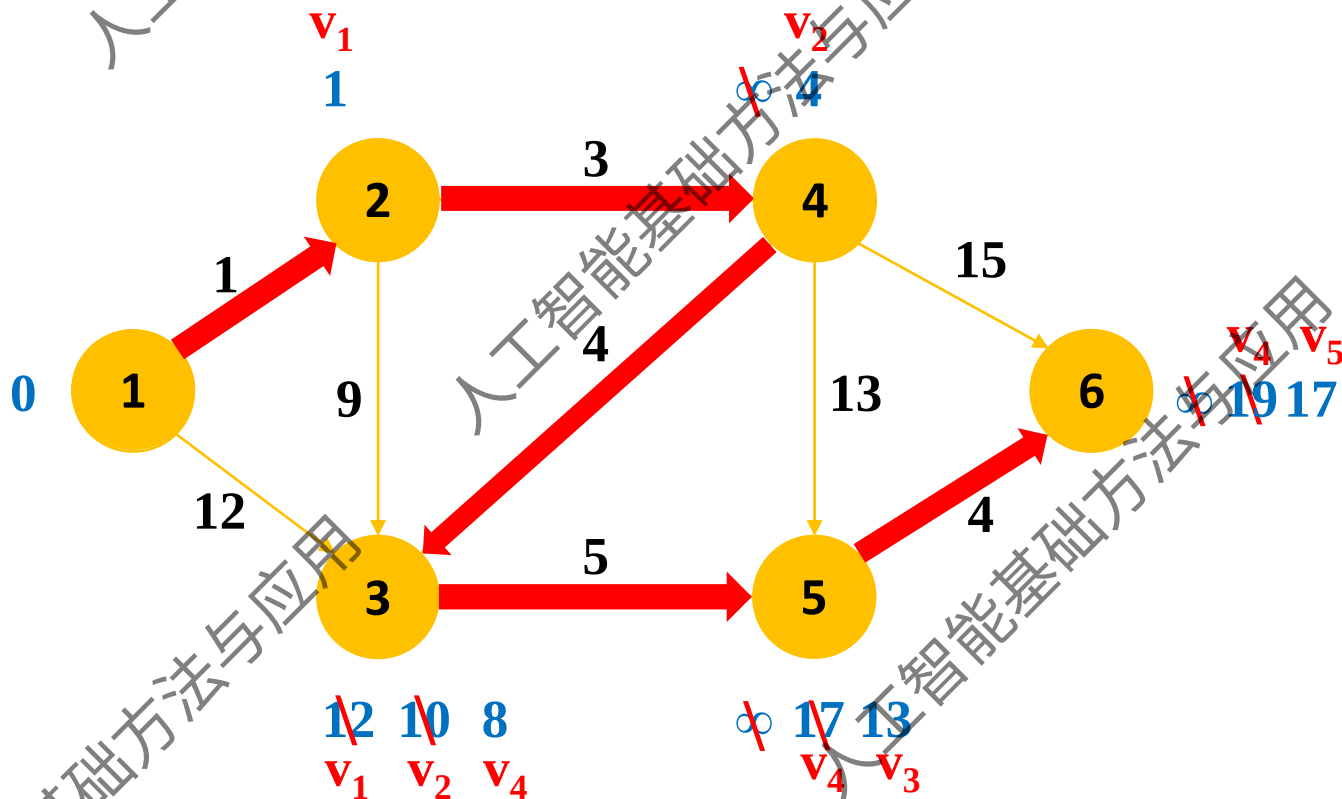
Final Path:

s a t

Closed Set:

s b c a



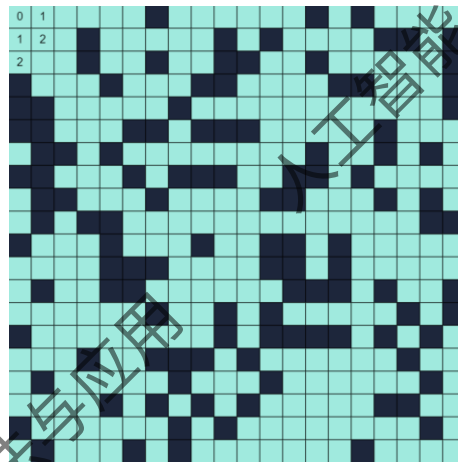


Dijkstra的可扩展性问题

- Dijkstra 每次用最小堆维护“当前最有希望的节点”需要：
 - 取出当前距离最小的节点
 - 更新相邻节点的距离
 - 必要时重新调整堆
 - 如果使用二叉最小堆，时间复杂度通常是：

$$O((V + E) \log V)$$

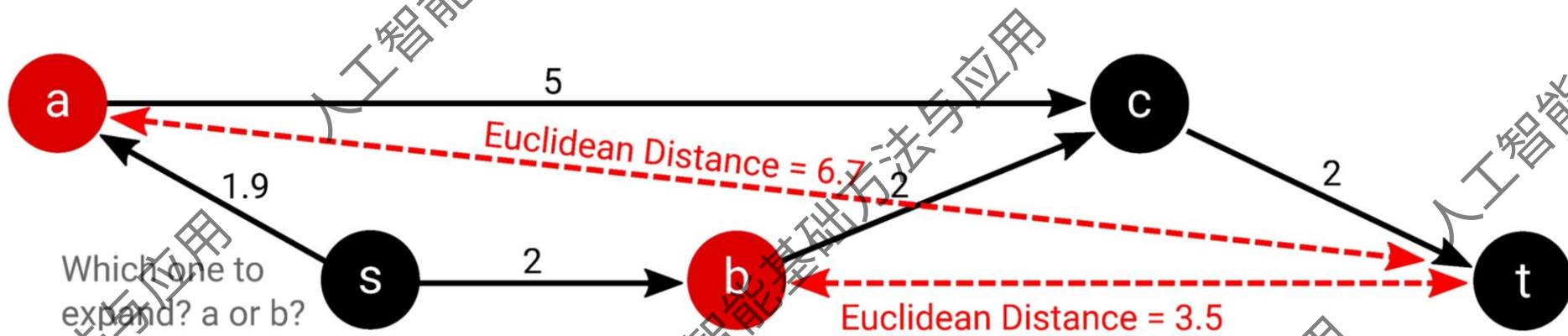
- BFS “按层扩散”
- Dijkstra “按成本扩散”



纽约市道路地图

- 54837个顶点
- 140497条边

欧式距离启发



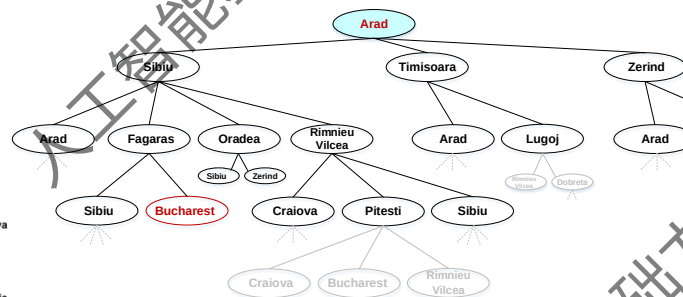
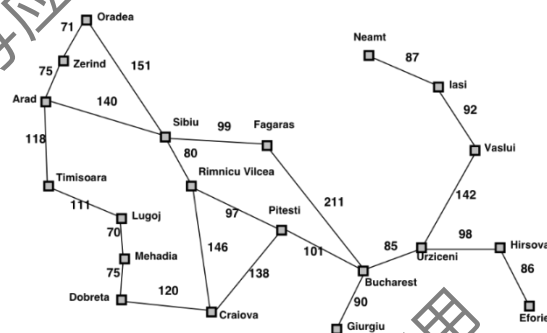
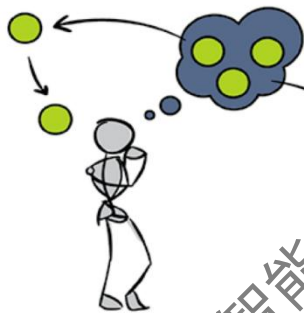
- 欧式距离启发方法利用平面图的几何结构进行路径规划
- 两个顶点之间的直线距离，可以作为实际路径距离的有效估计
- A*算法利用这一启发信息，优先扩展更有希望的顶点，从而提高规划效率
- 当启发函数不高估到达目标的实际剩余代价时，A*算法能够保证得到最优解

六、A*搜索算法



启发式搜索

- 启发：应用**特定的经验法则**或从经验衍生出来的论据，提高解决复杂问题的效率
- 启发搜索：利用启发方式获得的**领域知识**，通过**限定搜索深度**或者**限定搜索宽度**来缩小问题空间，避开没有结果的搜索路径
 - 引导搜索沿着一条**最可能的路径**到达解
 - 不是严格按照DFS、BFS预先确定方式或方法，“凭经验”或“试错”来确定要下一步扩展的节点



启发函数

- 启发函数：启发信息通常被形式化为一个关于节点的函数 $h(n)$ ，评估当前状态
- 启发是一个用于估计状态与目标状态的接近程度的函数（例如，曼哈顿距离，欧几里得距离等，距离越短，目标越近），并引入搜索过程中



$$h(n) \geq 0$$

$h(n)$ 越小，表示当前状态 n 越接近目标状态

$h(n) = 0$ 表示已达到目标

- 启发搜索利用启发函数的值将问题状态的描述转化为问题解决程度的描述
- 构造和选择合适的启发函数是启发式搜索的关键

评价函数

用于评价节点重要程度的函数，其主要任务是确定节点的优先级程度

● 评价函数 $f(n)$ 的一般形式：

$$f(n) = g(n) + h(n)$$

从初始状态（节点）到当前状态（节点）已付出的代价



从当前状态（节点）到目标状态（节点）的代价（到达目标尚剩余的距离）



$f(n) = g(n)$ 等代价搜索，按照已付出的代价进行搜索（如广度优先搜索），具有完备性

$f(n) = h(n)$ 按照启发函数向最靠近目标的状态（节点）搜索（如贪婪搜索），不具有完备性

A*搜索

- 功能：在有权图中求解最短路径问题
 - 在有权图中给出起点出发，到达目标点所经过边的权重和最小的一条路径
- 输入：有权图、目标节点
- 输出：起点到目标点的路径
- 评价函数：

$$f^*(n) = g^*(n) + h^*(n)$$

其中, $g^*(n)$ ：从起始点到节点n的路径最低代价

$h^*(n)$ ：从节点n到目标节点的最低代价路径的代价

$f^*(n)$ ：从起始点出发、经过节点n、到达目标节点的最佳路径的估计总代价

- 启发函数 $h(n)$ 满足某些条件确保A*搜索的完备性、最优性：

1) $g(n)$ 是对 $g^*(n)$ 的估计, 且 $g(n) > 0$

2) $h(n)$ 是 $h^*(n)$ 的下界, 即对于任意节点均有 $h(n) \leq h^*(n)$

A*搜索

Algorithm 2 Dijkstra(G, s, t)

```
1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s, 0)
3: while !open.isEmpty() do
4:   u, uCost  $\leftarrow$  open.pop()
5:   if u = t then
6:     return extractPath(u, predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u, v)
10:      if v  $\in$  open then
11:        if uCost + uvCost < open[v] then
12:          open[v]  $\leftarrow$  uCost + uvCost
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v, uCost + uvCost)
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
```

Algorithm 3 A*(G, s, t)

```
1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s, 0, h(s))
3: while !open.isEmpty() do
4:   u, uCost, uHeuristic  $\leftarrow$  open.pop() [based on cost+heuristic]
5:   if u = t then
6:     return extractPath(u, predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u, v)
10:      if v  $\in$  open then
11:        if uCost + uvCost + h(v) < fullCost(v) then
12:          open[v]  $\leftarrow$  uCost + uvCost, h(v)
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v, uCost + uvCost, h(v))
16:        predecessors[v]  $\leftarrow$  u
17:   closed.add(u)
```

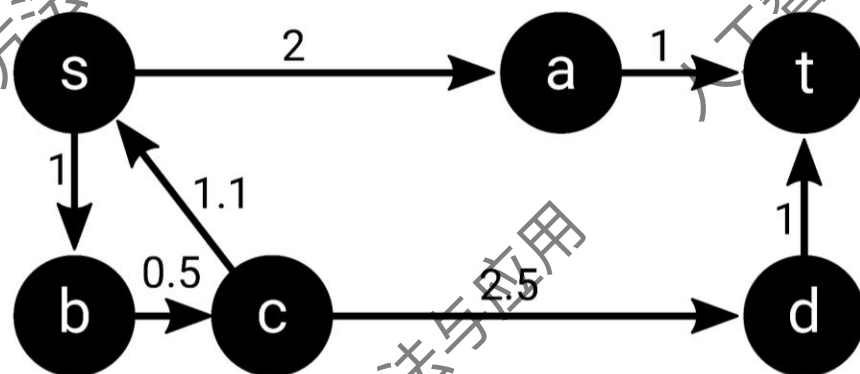


示例3

Algorithm 3 A*(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0,h(s))
3: while !open.isEmpty() do
4:   u, uCost, uHeuristic  $\leftarrow$  open.pop() [based on cost+heuristic]
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost + h(v) < fullCost(v) then
12:          open[v]  $\leftarrow$  uCost + uvCost, h(v)
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost, h(v))
16:        predecessors[v]  $\leftarrow$  u
17:      closed.add(u)
  
```



Open Min Heap:

Node	s
Cost + Heuristic	3

Closed Set:

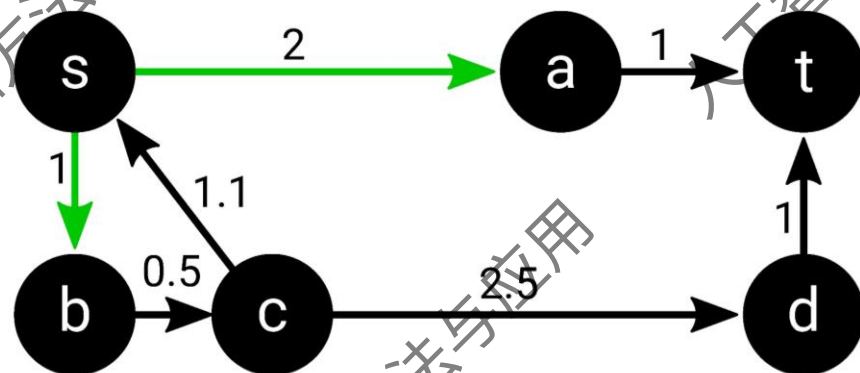
--

示例3

Algorithm 3 $A^*(G,s,t)$

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s, 0, h(s))
3: while !open.isEmpty() do
4:   u, uCost, uHeuristic  $\leftarrow$  open.pop() [based on cost+heuristic]
5:   if u = t then
6:     return extractPath(u, predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u, v)
10:      if v  $\in$  open then
11:        if uCost + uvCost + h(v) < fullCost(v) then
12:          open[v]  $\leftarrow$  uCost + uvCost, h(v)
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v, uCost + uvCost, h(v))
16:        predecessors[v]  $\leftarrow$  u
17:      closed.add(u)
  
```



Open Min Heap:

Node	a	b
Cost + Heuristic	3	4.2

Closed Set:

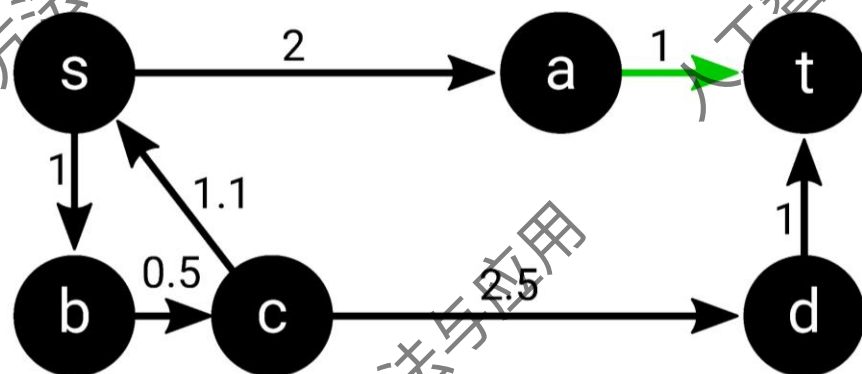
s

示例3

Algorithm 3 A*(G,s,t)

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0,h(s))
3: while !open.isEmpty() do
4:   u, uCost, uHeuristic  $\leftarrow$  open.pop() [based on cost+heuristic]
5:   if u = t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost + h(v) < fullCost(v) then
12:          open[v]  $\leftarrow$  uCost + uvCost, h(v)
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost, h(v))
16:        predecessors[v]  $\leftarrow$  u
17:      closed.add(u)
  
```



Open Min Heap:

Node	t	b
Cost + Heuristic	3	4.2

Closed Set:

s	a
---	---

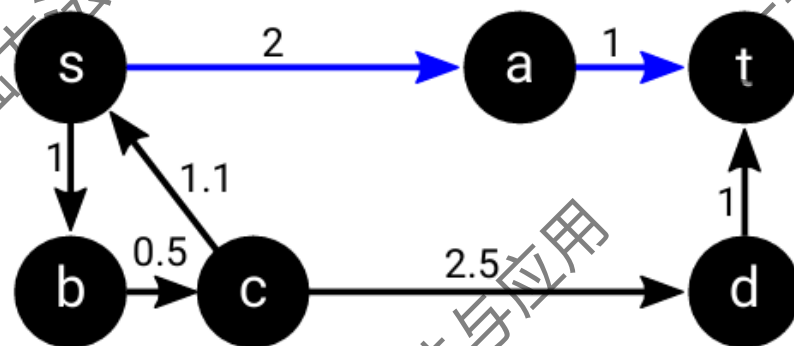


示例3

Algorithm 3 $A^*(G,s,t)$

```

1: open  $\leftarrow$  MinHeap(), closed  $\leftarrow$  Set(), predecessors  $\leftarrow$  Dict()
2: open.push(s,0,h(s))
3: while !open.isEmpty() do
4:   u, uCost, uHeuristic  $\leftarrow$  open.pop() [based on cost+heuristic]
5:   if u  $\neq$  t then
6:     return extractPath(u,predecessors)
7:   for all v  $\in$  u.successors() do
8:     if v  $\notin$  closed then
9:       uvCost  $\leftarrow$  edgeCost(u,v)
10:      if v  $\in$  open then
11:        if uCost + uvCost + h(v) < fullCost(v) then
12:          open[v]  $\leftarrow$  uCost + uvCost, h(v)
13:          predecessors[v]  $\leftarrow$  u
14:      else
15:        open.push(v,uCost + uvCost, h(v))
16:        predecessors[v]  $\leftarrow$  u
17:      closed.add(u)
  
```



Final Path:

s a t

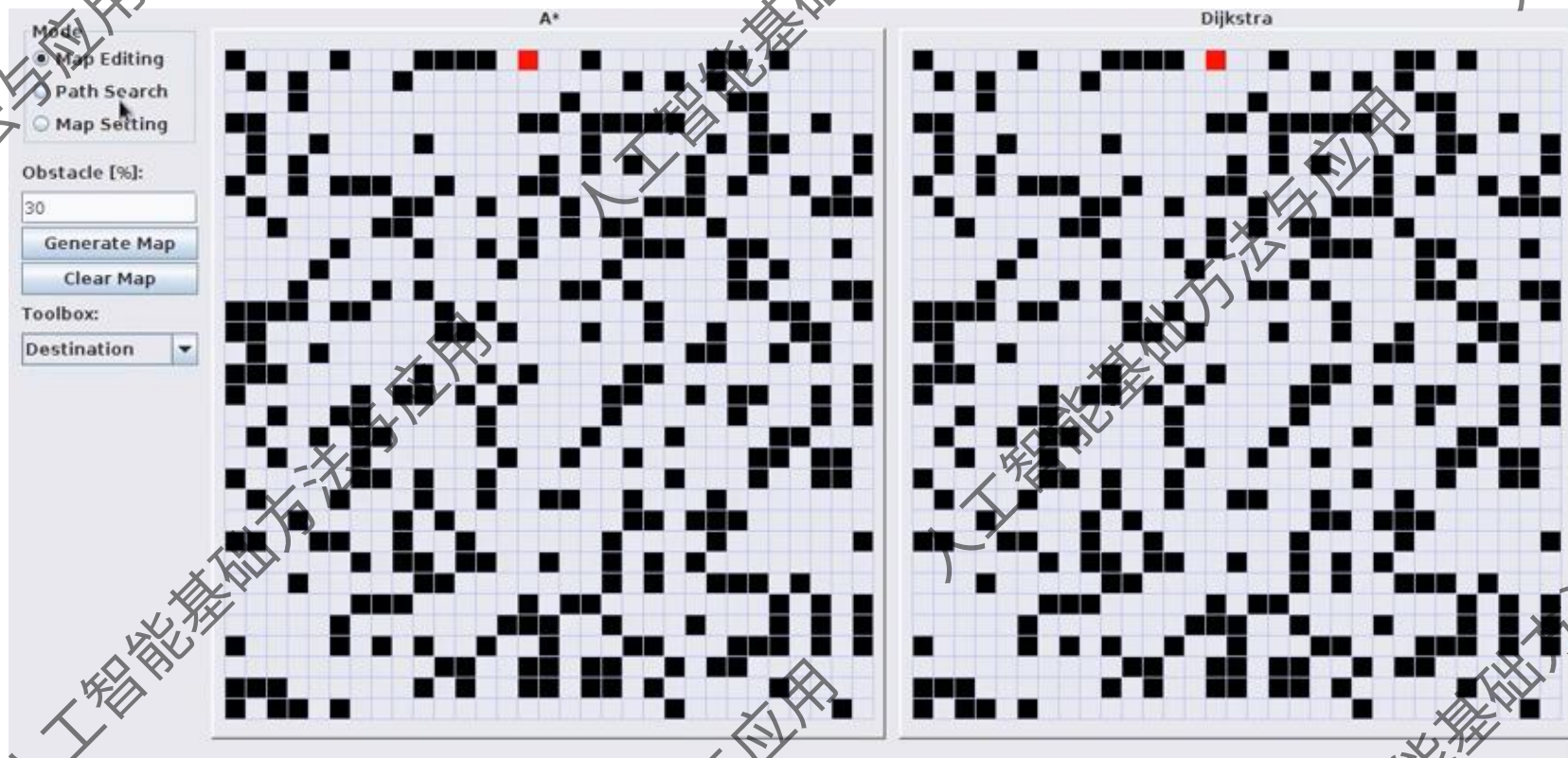
Closed Set:

s a



A*算法总结

- 优点：以高效率计算得出相对较短的路径（广度优先搜索，Dijkstra是遍历）
- 缺点：并不是绝对最短的路径
- 如果 $h(n)$ 是0，则只有 $g(n)$ 起作用，此时A*演变成Dijkstra算法



<https://github.com/kevinwang1975/PathFinder>



启发式搜索特点

- 优点：可缩小搜索范围，减少盲目性，有效提高搜索效率，更容易解决复杂的问题
- 关键：
 - 1) 具有启发函数：估计当前节点距离达到目标节点还需要多少代价（例如，在求最短路径的问题中，该函数为以相同的速度到达两个城市间的时间代价）
 - 2) 具有评价函数：根据搜索算法的不同，评价函数不同（例如，评价函数是用以选择最优的边缘结点等）
 - 3) 启发信息：是用以额外地帮助搜索的辅助信息（例如，各节点到目标结点的最短距离曼哈顿距离，欧几里得距离等，距离越短，目标越近）
- 缺点：
 - 可能不会始终提供最佳解决方案，但肯定会在合理的时间内提供良好的解决方案
 - 在不同的启发搜索算法中，使用不同的启发式方法



盲目搜索与启发式搜索

- 盲目搜索（非启发式搜索）：

- 只按照预定的控制策略进行搜索，搜索过程中获得的信息并不改变策略
- 搜索总按照预定路线进行，没有考虑问题本身的特性，缺乏对问题求解的针对性
- 需求全方位的搜索，没有选择最优的搜索途径，具有盲目性，效率不高

- 启发式搜索（有信息搜索）：

- 根据问题本身的特性或搜索过程中产生的一些信息改变或调整搜索方向，向最有希望的方向搜索，加速问题求解并找到最优解



搜索算法的评价指标

完备性

Complete

求解能力

即使在对于任何随机输入至少存在任何解决方案的情况下，可以确保一个解决方案，不一定为最优解

最优性

Optimal

求解质量

在所有可用替代方案中提供最佳解决方案

时间复杂度

Time Complexity

求解资源消耗

通过搜索算法以及完成的最大节点数来完成任务所花费的时间度量

空间复杂度

Space Complexity

表示搜索过程中所需的最大存储空间，具体取决于问题的复杂度



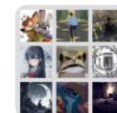
授课组联系方式

- 主讲：陈炜煌 助理教授（人工智能学院，曲江校区西一楼433）

Email: chenwh@xjtu.edu.cn

- 助教：刘帅宇 博士研究生（人工智能学院，曲江校区西一楼）

Email:
liushuaiyu@stu.xjtu.edu.cn



群聊：人工智能基础与应用I-06班



该二维码7天内(10月7日前)有效，重新进入将更新





为交大之光 为世界之光

2023.04



IAIR Est.
1986